

```

-- =====
-- 1. CATALOG & HIERARCHY MODULE
-- =====

CREATE TABLE category (
category_id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
description TEXT
);

CREATE TABLE sub_category (
sub_category_id SERIAL PRIMARY KEY,
category_id INT NOT NULL REFERENCES category(category_id) ON DELETE
CASCADE,
name VARCHAR(100) NOT NULL,
description TEXT
);

CREATE TABLE brand (
brand_id SERIAL PRIMARY KEY,
brand_name VARCHAR(100) NOT NULL,
manufacturer_details TEXT
);

CREATE TABLE product (
product_id SERIAL PRIMARY KEY,
sub_category_id INT NOT NULL REFERENCES sub_category(sub_category_id),
-- SET NULL: product remains even if brand goes out of business
brand_id INT REFERENCES brand(brand_id) ON DELETE SET NULL,
name VARCHAR(200) NOT NULL,
base_price DECIMAL(10, 2) NOT NULL CHECK (base_price > 0),
weight_volume VARCHAR(50)
-- Note: expiry_date belongs in inventory (per batch), not here
);

-- =====
-- 2. INVENTORY & WAREHOUSING MODULE
-- =====

```

```
CREATE TABLE warehouse (  
warehouse_id SERIAL PRIMARY KEY,  
name VARCHAR(100) NOT NULL,  
location TEXT NOT NULL,  
capacity_sqft DECIMAL(10, 2) CHECK (capacity_sqft > 0),  
-- Manager/in-charge employee (SET NULL if employee leaves)  
employee_id INT -- FK added after employee table  
);  
  
CREATE TABLE inventory (  
inventory_id SERIAL PRIMARY KEY,  
-- RESTRICT: preserve historical stock data  
product_id INT NOT NULL REFERENCES product(product_id),  
warehouse_id INT NOT NULL REFERENCES warehouse(warehouse_id),  
batch_number VARCHAR(100) NOT NULL,  
mfg_date DATE,  
expiry_date DATE,  
current_quantity INT DEFAULT 0 CHECK (current_quantity >= 0),  
last_updated DATE NOT NULL DEFAULT CURRENT_DATE,  
CHECK (expiry_date IS NULL OR mfg_date IS NULL OR expiry_date >=  
mfg_date)  
);  
  
CREATE TABLE reorder_level (  
reorder_id SERIAL PRIMARY KEY,  
-- CASCADE: drop rules if product or warehouse is removed  
product_id INT NOT NULL REFERENCES product(product_id) ON DELETE  
CASCADE,  
warehouse_id INT NOT NULL REFERENCES warehouse(warehouse_id) ON DELETE  
CASCADE,  
min_quantity INT NOT NULL CHECK (min_quantity > 0),  
target_quantity INT NOT NULL,  
CHECK (target_quantity > min_quantity)  
);  
  
CREATE TABLE stock_transaction (  
transaction_id SERIAL PRIMARY KEY,  
inventory_id INT NOT NULL REFERENCES inventory(inventory_id),  
product_id INT NOT NULL REFERENCES product(product_id),
```

```
-- Added TRANSFER back from V1; V2 dropped it
transaction_type VARCHAR(20) NOT NULL CHECK (transaction_type IN
('IN', 'OUT', 'TRANSFER')),
quantity INT NOT NULL CHECK (quantity > 0),
transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
supplier_id INT, -- FK added after supplier table
remarks TEXT
);
```

```
-- =====
-- 3. HR & SECURITY MODULE
-- =====
```

```
CREATE TABLE role (
role_id SERIAL PRIMARY KEY,
role_name VARCHAR(50) UNIQUE NOT NULL,
-- INT for programmatic access control comparisons
access_level INT NOT NULL CHECK (access_level >= 0)
);
```

```
CREATE TABLE shift (
shift_id SERIAL PRIMARY KEY,
shift_name VARCHAR(50) NOT NULL,
-- TIME (not TIMESTAMP) since shifts repeat daily
start_time TIME NOT NULL,
end_time TIME NOT NULL,
CHECK (start_time < end_time)
);
```

```
CREATE TABLE employee (
employee_id SERIAL PRIMARY KEY,
role_id INT NOT NULL REFERENCES role(role_id),
shift_id INT NOT NULL REFERENCES shift(shift_id),
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50),
phone VARCHAR(20),
hire_date DATE,
salary DECIMAL(10, 2) CHECK (salary >= 0)
);
```

```

CREATE TABLE user_login (
login_id SERIAL PRIMARY KEY,
-- CASCADE: remove login if employee is deleted
employee_id INT UNIQUE NOT NULL REFERENCES employee(employee_id) ON
DELETE CASCADE,
username VARCHAR(50) UNIQUE NOT NULL,
-- Explicitly named "password_hash" for clarity
password_hash VARCHAR(255) NOT NULL,
last_login_time TIMESTAMP,
is_active BOOLEAN DEFAULT TRUE
);

-- Now add warehouse manager FK (after employee table exists)
ALTER TABLE warehouse
ADD CONSTRAINT fk_warehouse_employee
FOREIGN KEY (employee_id) REFERENCES employee(employee_id) ON DELETE
SET NULL;

-- =====
-- 4. SUPPLY CHAIN & PURCHASING MODULE
-- =====

CREATE TABLE supplier (
supplier_id SERIAL PRIMARY KEY,
name VARCHAR(200) NOT NULL,
contact_person VARCHAR(100),
phone VARCHAR(20) UNIQUE NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
address TEXT NOT NULL,
city VARCHAR(100) NOT NULL,
CHECK (phone ~ '^[0-9]+$')
);

-- Now add stock_transaction supplier FK (after supplier table exists)
ALTER TABLE stock_transaction
ADD CONSTRAINT fk_stock_supplier
FOREIGN KEY (supplier_id) REFERENCES supplier(supplier_id) ON DELETE
SET NULL;

```

```

CREATE TABLE purchase_order (
po_id SERIAL PRIMARY KEY,
supplier_id INT NOT NULL REFERENCES supplier(supplier_id),
employee_id INT NOT NULL REFERENCES employee(employee_id),
order_date DATE NOT NULL,
expected_delivery_date DATE NOT NULL,
CHECK (expected_delivery_date >= order_date),
status VARCHAR(50) DEFAULT 'Pending'
CHECK (status IN ('Pending', 'Delivered', 'Cancelled')),
total_cost DECIMAL(12, 2) CHECK (total_cost >= 0)
);

```

```

CREATE TABLE purchase_order_details (
-- Composite PK: natural, no surrogate needed
po_id INT NOT NULL REFERENCES purchase_order(po_id) ON DELETE CASCADE,
product_id INT NOT NULL REFERENCES product(product_id),
order_quantity INT NOT NULL CHECK (order_quantity > 0),
received_quantity INT DEFAULT 0 CHECK (received_quantity >= 0),
unit_cost DECIMAL(10, 2) NOT NULL CHECK (unit_cost > 0),
PRIMARY KEY (po_id, product_id)
);

```

```

CREATE TABLE purchase_return (
return_id SERIAL PRIMARY KEY,
po_id INT NOT NULL REFERENCES purchase_order(po_id),
product_id INT NOT NULL REFERENCES product(product_id),
employee_id INT NOT NULL REFERENCES employee(employee_id),
return_quantity INT NOT NULL CHECK (return_quantity > 0),
reason VARCHAR(255),
refund_amount DECIMAL(10, 2) CHECK (refund_amount >= 0),
return_date DATE DEFAULT CURRENT_DATE
);

```

```

-- =====
-- 5. CUSTOMER & LOYALTY MODULE
-- =====

```

```

CREATE TABLE customer (

```

```

customer_id SERIAL PRIMARY KEY,
phone_number VARCHAR(20) UNIQUE NOT NULL,
name VARCHAR(100),
email VARCHAR(100),
registration_date DATE DEFAULT CURRENT_DATE,
CHECK (email IS NULL OR email ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'),
CHECK (phone_number ~ '^[0-9]+$')
);

-- Tier definitions (standalone, not per-customer)
CREATE TABLE membership (
membership_id SERIAL PRIMARY KEY,
tier_name VARCHAR(50) NOT NULL UNIQUE,
points_multiplier DECIMAL(5, 2) DEFAULT 1.0 CHECK (points_multiplier > 0),
minimum_spend_required DECIMAL(10, 2) DEFAULT 0.0 CHECK (minimum_spend_required >= 0)
);

CREATE TABLE membership_benefits (
benefit_id SERIAL PRIMARY KEY,
-- CASCADE: remove benefits if tier is removed
membership_id INT NOT NULL REFERENCES membership(membership_id) ON DELETE CASCADE,
benefit_description TEXT NOT NULL
);

CREATE TABLE loyalty_account (
account_id SERIAL PRIMARY KEY,
-- CASCADE: remove loyalty account if customer is removed
customer_id INT UNIQUE NOT NULL REFERENCES customer(customer_id) ON DELETE CASCADE,
membership_id INT NOT NULL REFERENCES membership(membership_id),
total_points_balance INT DEFAULT 0 CHECK (total_points_balance >= 0),
date_of_creation DATE DEFAULT CURRENT_DATE
);

-- =====

```

```
-- 6. SALES & POS CHECKOUT MODULE
```

```
-- =====
```

```
CREATE TABLE sales_bill (  
  bill_id SERIAL PRIMARY KEY,  
  -- SET NULL: keep bill history even if customer is deleted  
  customer_id INT REFERENCES customer(customer_id) ON DELETE SET NULL,  
  employee_id INT NOT NULL REFERENCES employee(employee_id),  
  terminal_id VARCHAR(50),  
  bill_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  sub_total DECIMAL(10, 2) NOT NULL CHECK (sub_total >= 0),  
  tax_amount DECIMAL(10, 2) NOT NULL CHECK (tax_amount >= 0),  
  grand_total DECIMAL(10, 2) NOT NULL CHECK (grand_total > 0),  
  payment_status VARCHAR(50) NOT NULL CHECK (payment_status IN ('Paid',  
    'Pending', 'Failed', 'Refunded'))  
);
```

```
CREATE TABLE sales_bill_details (  
  -- Composite PK: one row per product per bill  
  bill_id INT NOT NULL REFERENCES sales_bill(bill_id) ON DELETE CASCADE,  
  inventory_id INT NOT NULL REFERENCES inventory(inventory_id),  
  quantity INT NOT NULL CHECK (quantity > 0),  
  selling_price DECIMAL(10, 2) NOT NULL CHECK (selling_price > 0),  
  discount_applied DECIMAL(10, 2) DEFAULT 0.0 CHECK (discount_applied >= 0),  
  return_period INT CHECK (return_period >= 0), -- days  
  PRIMARY KEY (bill_id, inventory_id)  
);
```

```
CREATE TABLE payment (  
  payment_id SERIAL PRIMARY KEY,  
  -- CASCADE: remove payment if bill/PO is deleted  
  bill_id INT REFERENCES sales_bill(bill_id) ON DELETE CASCADE,  
  purchase_id INT REFERENCES purchase_order(po_id) ON DELETE CASCADE,  
  payment_method VARCHAR(50) NOT NULL  
  CHECK (payment_method IN ('Cash', 'Card', 'UPI', 'Net Banking')),  
  amount_paid DECIMAL(10, 2) NOT NULL CHECK (amount_paid > 0),  
  payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  transaction_reference VARCHAR(100),
```

```

status VARCHAR(50) DEFAULT 'Completed'
CHECK (status IN ('Paid', 'Pending', 'Failed', 'Refunded')),
-- XOR: must link to either a bill OR a purchase, not both, not
neither
CHECK (
(bill_id IS NOT NULL AND purchase_id IS NULL) OR
(bill_id IS NULL AND purchase_id IS NOT NULL)
)
);

CREATE TABLE receipt (
receipt_id SERIAL PRIMARY KEY,
-- CASCADE: remove receipt if bill or payment is deleted
bill_id INT UNIQUE NOT NULL REFERENCES sales_bill(bill_id) ON DELETE
CASCADE,
payment_id INT UNIQUE NOT NULL REFERENCES payment(payment_id) ON
DELETE CASCADE,
receipt_number VARCHAR(100) UNIQUE NOT NULL,
receipt_date DATE DEFAULT CURRENT_DATE
);

CREATE TABLE sales_return (
sales_return_id SERIAL PRIMARY KEY,
bill_id INT NOT NULL REFERENCES sales_bill(bill_id),
inventory_id INT NOT NULL REFERENCES inventory(inventory_id),
return_quantity INT NOT NULL CHECK (return_quantity > 0),
reason VARCHAR(255),
refund_amount DECIMAL(10, 2) NOT NULL CHECK (refund_amount >= 0),
return_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- 7. LOYALTY TRANSACTIONS MODULE
-- =====

CREATE TABLE loyalty_transaction (
lt_id SERIAL PRIMARY KEY,
-- CASCADE: remove transactions if loyalty account is deleted

```

```

account_id INT NOT NULL REFERENCES loyalty_account(account_id) ON
DELETE CASCADE,
-- SET NULL: keep loyalty history even if bill is deleted
bill_id INT REFERENCES sales_bill(bill_id) ON DELETE SET NULL,
transaction_type VARCHAR(50) NOT NULL CHECK (transaction_type IN
('Earned', 'Redeemed')),
points INT NOT NULL CHECK (points > 0),
transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- 8. MARKETING MODULE
-- =====

CREATE TABLE festival (
festival_id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
start_date DATE NOT NULL,
end_date DATE NOT NULL,
CHECK (end_date >= start_date)
);

CREATE TABLE promotion (
promo_id SERIAL PRIMARY KEY,
-- SET NULL: keep promotions even if festival ends/deleted
festival_id INT REFERENCES festival(festival_id) ON DELETE SET NULL,
name VARCHAR(100) NOT NULL,
discount_type VARCHAR(50) NOT NULL CHECK (discount_type IN
('Percentage', 'Flat')),
discount_value DECIMAL(10, 2) NOT NULL CHECK (discount_value > 0),
start_date DATE NOT NULL,
end_date DATE,
CHECK (end_date IS NULL OR end_date >= start_date),
is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE product_promotion (
-- Composite PK: natural junction table

```

```
promo_id INT NOT NULL REFERENCES promotion(promo_id) ON DELETE
CASCADE,
product_id INT NOT NULL REFERENCES product(product_id) ON DELETE
CASCADE,
PRIMARY KEY (promo_id, product_id)
);

CREATE TABLE product_demand (
demand_id SERIAL PRIMARY KEY,
product_id INT UNIQUE NOT NULL REFERENCES product(product_id) ON
DELETE CASCADE,
festival_id INT REFERENCES festival(festival_id) ON DELETE SET NULL,
demand_score DECIMAL(5, 2),
units_sold INT CHECK (units_sold >= 0),
demand_level VARCHAR(50),
season_tag VARCHAR(50),
last_calculated_date DATE DEFAULT CURRENT_DATE
);
```